# LINGUAGEM DE PROGRAMAÇÃO I

## STRINGS

UNIVERSIDADE FEDERAL DO ACRE

PROF.: CLECIO ELIAS

EMAIL.:CLECIO.SILVA@UFAC.BR

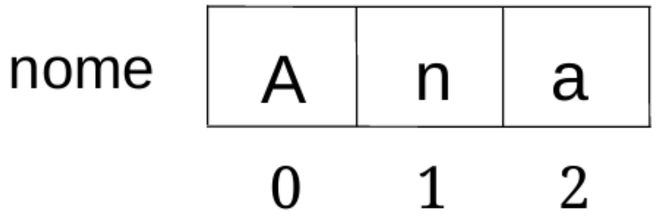
# Strings

## Definição

Uma string é uma **sequência** de caracteres utilizada para **representar textos**. Em Python, strings são objetos do tipo **str** e possuem tamanho e valor **imutáveis**.

```
nome = "Ana"

print(nome)
```




Exemplo de uma cadeia de 3 caracteres e valor definido

# Strings

## Strings são imutáveis

Strings são **imutáveis**, ou seja, seus caracteres não podem ser alterados individualmente após a criação do objeto.

```
texto = "Alô Mundo!"


texto[0] = "@"
```

Isso gera um **TypeError**.

Para **modificar** uma string, é necessário criar uma **nova** string.

```
texto = "Alô Mundo!"
texto = "@" + texto[1:]


print(texto)
```

# Strings

## Strings e listas

Embora strings sejam imutáveis, seus caracteres podem ser manipulados individualmente quando a string é **convertida** para uma lista de caracteres.

```
L = list("Alô Mundo")

L[0] = "a"

texto = "".join(L)

print(texto)
```

```
"Alô Mundo"
     ↓ list()
['A','l','ô',' ','M','u','n','d','o']
     ↓ alteração
['a','l','ô',' ','M','u','n','d','o']
     ↓ join()
"alô Mundo"
```

- **list()** → transforma a string em uma lista de caracteres.
- **join()** → reúne os elementos da lista em uma string.

# Strings

## join()

O método **join()** combina os elementos de um iterável em uma **única string**, utilizando a string que o chamou como separador.

```
palavras = ["Python", "é", "mágico!"]

frase = " ".join(palavras)

print(frase)
```

Saida:
Python é mágico!

```
frase = "-".join(["fla", "men", "go"])

print(frase)
```

Saida:
fla-men-go

# Strings

## Declaração e comprimento

Strings podem ser delimitadas por aspas **simples** ou **duplas**.

```
x = 'Tiaguenta'
y = "com batata frita"
```

Também podemos obter o número de caracteres utilizando **len()**:

```
texto = "ABCD"


print(len(texto))
```

```
ultimo = texto[len(texto) - 1]
```

Uso do **len()** de forma prática

# Strings

## Particularidade do Python

Python **armazena** strings de mesmo conteúdo no mesmo endereço de memória, um conceito conhecido como "**String Interning**".

- Quando uma string é criada, o Python verifica se já existe uma string com o mesmo conteúdo na memória.
- Se encontrar, a nova variável simplesmente apontará para o objeto de string já existente, sem alocar um novo local na memória

**Vantagens:**

– **Economia de memória:** Ao evitar a criação de múltiplos objetos para o mesmo conteúdo, o Python reduz significativamente o consumo de memória.

– **Melhor desempenho:** O uso de um único objeto de string para o mesmo conteúdo agiliza operações de **comparação**, pois o Python não precisa comparar cada caractere individualmente; ele apenas verifica se os **endereços de memória** são os mesmos.

# Strings

## Comparação de Strings

Strings podem ser comparadas utilizando **operadores relacionais**. A comparação é realizada **lexicograficamente**, considerando a ordem dos caracteres.

- **== igual**
- **!= diferente**
- **< menor**
- **> maior**
- **<= menor ou igual**
- **>= maior ou igual**

```
x = "ABCD"
y = "ABCZ"

print(x < y)
print(x != "DCBA")
```

```
a = ["Python"]
b = ["Python"]

print(a == b)  # True
print(a is b)  # False
```

***is** verifica se duas variáveis se referem ao mesmo objeto, não se possuem apenas o mesmo conteúdo.

# Strings

## Fatiamento de Strings

O fatiamento permite obter uma **nova** string a **partir** de uma parte da string **original**.

```
string[início:fim]
```

```
nome = "Manoel Limeira de Lima Júnior"

sobrenome = nome[7:14]

print(sobrenome)
```

# Strings

## Fatiamento: índices omitidos e negativos

No fatiamento, os **limites** podem ser omitidos e também podem ser utilizados índices negativos.

```
nome = "Manoel Limeira de Lima Júnior"


print(nome[:6])
print(nome[7:])
print(nome[-6:])
```

nome[-1] → último caractere
nome[-2] → penúltimo caractere
⋮

# Strings

## Fatiamento com incremento

O fatiamento pode receber um **terceiro** parâmetro, chamado **incremento**, que determina o intervalo entre os caracteres selecionados.

```
string[início:fim:incremento]
```

```
nome = "Manoel Limeira de Lima"


print(nome[::2])
print(nome[1::3])
print(nome[::-1])
```

**Saidas:**

Mne ier eLm

aeLeaei

amiL ed ariemiL leonaM

*Um incremento de -1 percorre a string de trás para frente, permitindo inverter uma string.

# Strings

## Buscando texto com **find()**

O método **find()** procura uma substring e retorna o índice de sua primeira ocorrência. Caso ela não seja encontrada, retorna -1.

```
string.find(substring, início, fim)
```

Os parâmetros início e fim são opcionais.

```
nome = "Manoel Limeira de Lima"


print(nome.find("Limeira"))
print(nome.find("a"))
```

# Strings

## Principais métodos de Strings

Python disponibiliza diversos métodos para **manipular**, **pesquisar** e **analisar** strings.

| | |
|---|---|
| **split()** | Divide a string em partes |
| **replace()** | Substitui ocorrências |
| **count()** | Conta ocorrências |
| **upper()** | Converte para maiúsculas |
| **lower()** | Converte para minúsculas |
| **strip()** | Remove espaços nas extremidades |

# Strings

## **split()** e replace()

Divide uma string em partes e retorna uma lista de strings.

```
frase = "Python é muito legal"

palavras = frase.split()

print(palavras)
```

Saida:
["Python", "é", "muito", "legal"]

### replace()

Retorna uma nova string substituindo todas as ocorrências de uma substring por outra.

```
texto = "Eu gosto de Java"

novo = texto.replace("Java", "Python")

print(novo)
```

# Strings

## count(), upper(), lower() e strip()

```python
texto = "   Python   "

print(texto.count("t"))
print(texto.upper())
print(texto.lower())
print(texto.strip())
```

- **count()** → conta ocorrências
- **upper()** → MAIÚSCULAS
- **lower()** → minúsculas
- **strip()** → remove espaços no início e no fim

*Esses métodos retornam uma nova string; a string original não é modificada.

# Strings

## Métodos de validação

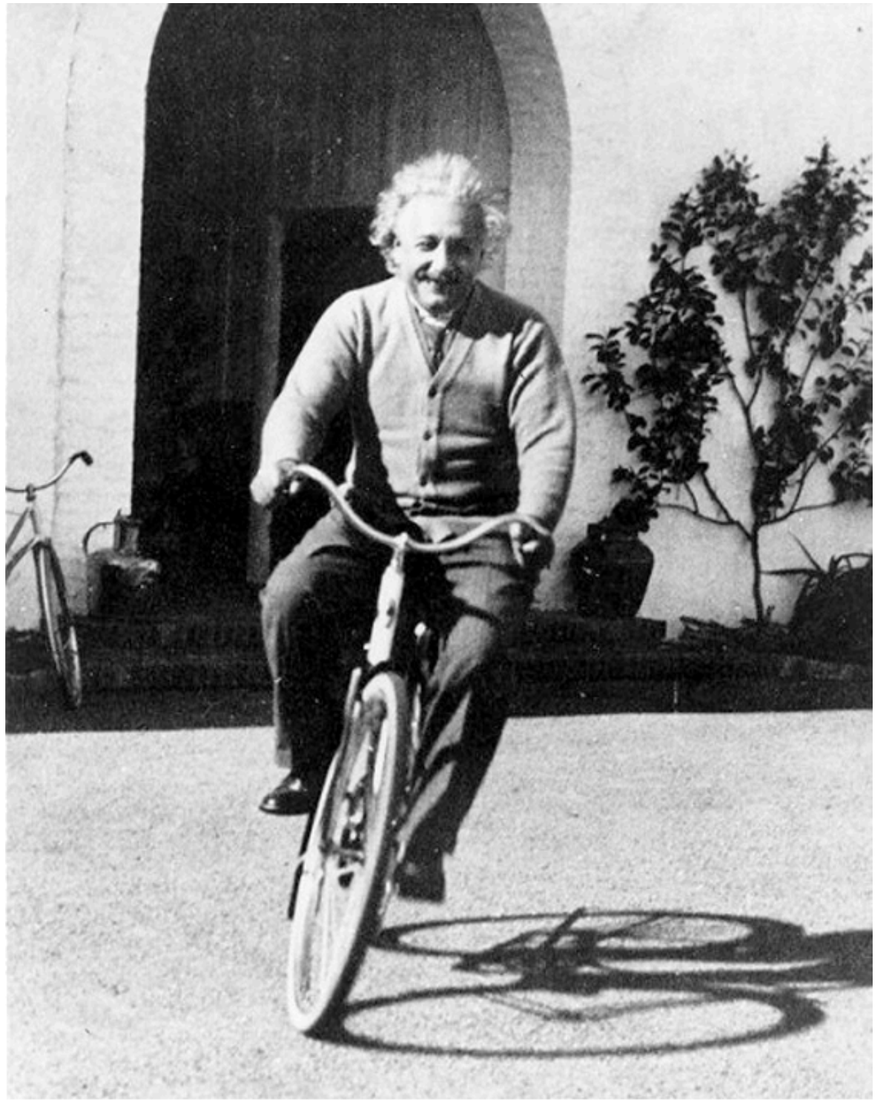
Métodos de validação permitem verificar determinadas características de uma string e retornam um valor booleano: **True** ou **False**.

```
print("Python".isalpha())
print("123".isdigit())
print("Python123".isalnum())
print("python".islower())
print("PYTHON".isupper())
print("    ".isspace())
```

→ Retorna True se a string não estiver **vazia** e se todos os caracteres são **letras**, **números** ou uma **combinação deles**

Esses métodos são úteis para validar dados de entrada antes de processá-los.

# Exercícios - URI



“A vida é como andar de bicicleta. Para manter o equilíbrio, é preciso se manter em movimento”.

- Einstein

**Lista de exercicios!**

# Conteudos a seguir - programação

| | |
|---|---|
| Strings + Lista exercicio (extra) Fecha N1 | - 12/09 Sábado |
| Coleções (Tuplas e etc) | - 17/09 Quinta |
| Funções (+Func anônimas) + Lista exercicio | - 19/09 Sábado |
| List Comprehension | - 24/09 Quinta |
| Iteradores e func geradoras + Lista exercicio | - 26/09 Sábado |
| Exceções | - 08/10 Quinta |
| Extra (Prova Final caso necessario) + Lista exercicio | - 10/10 Sábado |

Próximas listas (Nota completa da N2):

14/09 - 19/09

21/09 - 26/09

27/09 - 10/09